mCOLD — สิ่งที่ฝั่ง Server ต้องทำ (MQTT)

เวอร์ชัน 0.15 (ฉบับร่าง) · 2026-10-12 · firmware 0.7.0-dev.52

เอกสารนี้สรุปสำหรับทีม server: กล่อง mCOLD ส่งอะไรขึ้นมา, server ต้องตอบอะไรกลับ, และเรื่องที่ต้องตัดสินใจร่วมกัน ฝั่งกล่องทำเสร็จและเชื่อมต่อ broker ของทีมได้แล้ว **สิ่งที่ยังขาดคือ ACK จาก server** รายละเอียดเต็มอยู่ใน `firmware/PROTOCOL.md` ข้อ 6

1. สิ่งที่ต้องทำ เรียงตามความสำคัญ

#	เรื่อง	ถ้ายังไม่ทำ
1	**ส่ง ACK ทุกก้อน** หลังเก็บแถวลงฐานข้อมูลแล้ว (ข้อ 4)	กล่องส่งก้อนเดิมซ้ำไม่จบ เว้นระยะนานขึ้นถึง 4 ชั่วโมง และ trip ไม่มีวันขึ้น "ส่งแล้ว"
2	**ACK ให้เร็ว** ภายใน 1–2 วินาที	ทุกวินาทีที่รอคือวิทยุ Wi-Fi เปิดค้างในทุกกล่อง กินแบตโดยตรง
3	**เก็บแบบไม่ซ้ำ** โดยใช้ (trip_id, seq) เป็น key	กล่องส่งซ้ำทุกครั้งที่ไม่ได้ ACK ข้อมูลจะซ้ำในฐานข้อมูล
4	**เปิด TLS (8883) และ login แยกต่อเครื่อง**	ตอนนี้ข้อมูลและรหัสผ่านวิ่งแบบไม่เข้ารหัส และใครมี login เดียวกันก็ปลอม ACK ได้ ทำให้กล่องลบข้อมูลที่ยังไม่ถึง server
5	ส่ง OTA ตามข้อ 8: publish ชื่อไฟล์แบบ **retain** แล้วลบเมื่อได้ `ok`	กล่องที่ใช้แบตหลับอยู่จะไม่เห็นคำสั่ง
6	**ประกอบ trip จากก้อน** (`part` / `parts` / `last`, ข้อ 3)	ข้อมูล trip ไม่ครบหรือเรียงผิด
7	**ดู `status` ระหว่าง trip** (อุณหภูมิล่าสุด, alarm, `alarms_raised`, `last_alarm`)	ระหว่าง trip ไม่เห็นอะไรเลย เพราะแถวขึ้นหลังจบ trip
9	ตั้งค่ากล่องจาก server ด้วยเอกสารตั้งค่า (ข้อ 9) — `rev` ต้องเพิ่มทุกครั้ง	ส่งซ้ำ rev เดิม กล่องจะไม่ทำอะไร
8	**รับไฟล์ CSV จากแอป** (ปุ่ม "จบ trip และส่งข้อมูล") — ต้องกำหนดช่องทาง (ข้อ 7)	แอปส่ง trip ขึ้น server เองไม่ได้ ต้องรอกล่องเจอ Wi-Fi

2. การเชื่อมต่อ

- client id = รหัสเครื่อง (`sn`) ตามฉลาก เช่น `mCDV1-L0169-1069-001` (ตัวพิมพ์เล็ก-ใหญ่มีผล) กล่องที่ยังไม่ได้ตั้ง SN จากโรงงานจะใช้ชื่อชั่วคราว `MCOLD-xxxx`
- broker และ login ตั้งในเครื่องแต่ละกล่อง ไม่อยู่ใน firmware หรือ git
- แบบเดียวกับ eTEMP: **ที่กล่อง publish อยู่ใต้** `mcold/<sn>/` และ **ที่กล่อง subscribe (server publish) อยู่ใต้** `mcold/v1/<sn>/` (`v1` คือเวอร์ชันของคำสั่งที่กล่องเข้าใจ)
- สรุปทุก topic อยู่ที่ `mqtt-topics.md`

Topic	ทิศทาง	QoS	Retained	เนื้อหา
`mcold/<sn>/rec`	กล่อง → server	1	ไม่	ชุด record (ข้อ 3)
`mcold/v1/<sn>/ack`	**server → กล่อง**	1	ไม่	`{"trip_id":"<uuid>","upto":S}` (ข้อ 4)
`mcold/<sn>/status`	กล่อง → server	0	ใช่	สถานะล่าสุด (JSON เดียวกับ `GET_STATUS`)
`mcold/<sn>/online`	กล่อง → server	1	ใช่	`"1"` ตอนเชื่อมต่อ, `"0"` ตอนจบรอบหรือหลุด (last will)
`mcold/<sn>/ota/state`	กล่อง → server	1	ใช่	ผลการอัปเดต firmware (ข้อ 8)
`mcold/v1/<sn>/firmware`	**server → กล่อง**	1	**ใช่**	ชื่อไฟล์ firmware ที่จะให้อัปเดต (ข้อ 8)
`mcold/v1/<sn>/config`	**server → กล่อง**	1	**ใช่**	เอกสารตั้งค่า (ข้อ 9)
`mcold/<sn>/config/state`	กล่อง → server	1	ใช่	ผลการตั้งค่า (ข้อ 9)
`mcold/<sn>/settings`	กล่อง → server	1	ใช่	**ค่าตั้งทั้งหมดของกล่องตอนนี้** ส่งใหม่ทุกครั้งที่มีการแก้ (ข้อ 10)

รับข้อมูลทุกกล่องได้ด้วยการ subscribe `mcold/+/rec`

3. ข้อมูลที่กล่องส่งขึ้นมา

ทุก record เป็น **แถวเดียวกัน มี column ชุดเดียวกัน** ไม่ว่าจะเป็น sample หรือ event (ตกลงกันเมื่อ 5 ต.ค.) แต่ละแถวมีสถานะของกล่อง ณ ตอนนั้นครบ server เก็บลงตารางเดียวได้เลย หัวข้อนี้เป็น **column ของ MQTT เท่านั้น** ไฟล์ CSV (ที่แอปและไดรฟ์ USB ใช้) มีคอลัมน์มากกว่า ดู `../app/ble-protocol.md` ข้อ 6

```
{"sn":"mCDV1-L0169-1069-002","trip_id":"95518e06-1b3b-494f-b4c5-284ae94f5424","trip_date":"20261007","trip_number":2,"part":1,"parts":1,
"from":0,"to":6,"last":true,
"rows":[
{"trip_id":"95518e06-1b3b-494f-b4c5-
284ae94f5424","trip_date":"20261007","trip_number":2,"seq":1,"timestamp":1791143247,"event":"SAMPLE","temp":4.25,"tempmin":2,"tempmax":8,"alarm":"",
"timeok":true,"gnssstate":"last","latitude":13.7563,"longitude":100.5018,"motion":0,
"battery":97,"internet":"online"},
{"trip_id":"95518e06-1b3b-494f-b4c5-
284ae94f5424","trip_date":"20261007","trip_number":2,"seq":2,"timestamp":1791143248,"event":"ALARM_HIGH","temp":8.5,"tempmin":2,"tempmax":8,"alarm":"HIGH",
...}, ...]}
```

- **ส่งเฉพาะ trip ที่จบแล้ว** (ตกลง 5 ต.ค., `trip-data-flow.md`): ระหว่าง trip แถวอยู่ในกล่อง server ได้แค่ `status` · จบแล้วส่งเป็นก้อนละ 20 แถว trip เก่าก่อน
- `part` / `parts`: ก้อนที่ k มีแถว `seq` 20(k−1)…20k−1 เสมอ · `last: true` คือก้อนสุดท้าย (มี `TRIP_STOP`)
- trip ที่แอปส่งให้ server เองแล้ว (แอปเรียก `MARK_DELIVERED`) กล่องจะไม่ส่งซ้ำทาง Wi-Fi
- ระหว่าง trip `status` → `trip` มี `alarms_raised` และ `last_alarm` (ชนิด + เวลา) ใช้รู้ว่ามี alarm เกิดตอนกล่องออฟไลน์
- **key ของแต่ละแถวคือ (trip_id, seq)** — `trip_id` เป็น UUID ที่กล่องสร้างตอนเริ่ม trip (ตกลง 7 ต.ค.) ผูก trip กับกล่องด้วย `sn` ที่หัวชุด (และใน topic)

Column ของแถวใน MQTT (ตกลง 7 ต.ค.: ไม่มี `sn`, `utc`, `detail` เพราะ `sn` อยู่ที่หัวชุดและใน topic, `timestamp` เป็น Unix อยู่แล้ว และ `detail` ส่วนใหญ่ว่าง)

column	ชนิด	ความหมาย
`trip_id`	ข้อความ	UUID ของ trip ที่กล่องสร้าง
`trip_date`	ข้อความ	วันที่ท้องถิ่นตอนเริ่ม trip รูปแบบ YYYYMMDD (`"00000000"` ถ้าตอนเริ่มกล่องยังไม่รู้เวลา)
`trip_number`	ตัวเลข	เลขรันของ trip ในวันนั้น เริ่มที่ 1
`seq`	ตัวเลข	ลำดับแถวใน trip เริ่มที่ 0
`timestamp`	ตัวเลข / `null`	เวลาเป็นวินาที Unix (UTC) · `null` เมื่อ `timeok` เป็น `false`
`event`	ข้อความ	แถวนี้คืออะไร (ตารางถัดไป)
`temp`	ตัวเลข / `null`	อุณหภูมิ °C · `null` = probe อ่านไม่ได้
`tempmin` / `tempmax`	ตัวเลข	ช่วง alarm ของ trip นี้ (°C)
`alarm`	ข้อความ	alarm ที่ค้างอยู่ `HIGH`, `LOW`, `PROBE`, `BATTERY` หลายตัวคั่นด้วยขีดตั้ง เช่น `"HIGH|BATTERY"` · `""` = ไม่มี
`timeok`	true/false	`false` = กล่องไม่รู้เวลาตอนนั้น แถวนั้นมี `boot` และ `up_s` เพิ่มมาไว้เรียงลำดับ
`gnssstate`	ข้อความ	`fix` = ตำแหน่งใหม่, `last` = ตำแหน่งเก่า, `none` = ไม่มี
`latitude` / `longitude`	ตัวเลข / `null`	องศา · `null` เมื่อ `none`
`motion`	ตัวเลข	จำนวนครั้งที่ขยับตั้งแต่แถวก่อน
`battery`	ตัวเลข / `null`	แบต % · `null` ถ้าอ่านไม่ได้
`internet`	ข้อความ	ผลการส่งรอบล่าสุด: `online`, `wifi_only` (ต่อ Wi-Fi ได้แต่ไม่ถึง server), `offline`
`boot`	ตัวเลข	(เฉพาะแถวที่ `timeok` เป็น `false`) เลขการบูตของกล่อง
`up_s`	ตัวเลข	(เฉพาะแถวที่ `timeok` เป็น `false`) วินาทีนับจากบูต ใช้เรียงแถวในการบูตเดียวกัน

event	เกิดเมื่อ
`TRIP_START` / `TRIP_STOP`	เริ่ม / จบ trip
`SAMPLE`	ทุกรอบบันทึก (ค่าเริ่มต้น 5 นาที)
`ALARM_HIGH` / `ALARM_LOW`	อุณหภูมิเกิน `tempmax` / ต่ำกว่า `tempmin` นานกว่าเวลาที่ตั้ง (`alarm_dwell_s`)
`ALARM_PROBE`	อ่านอุณหภูมิไม่ได้เกิน 1 นาที
`BATTERY_LOW`	แบตต่ำกว่า 15%
`ALARM_CLEAR`	alarm หาย
`ALARM_ACK`	มีคนกดรับทราบ
`PROBE_FAULT` / `PROBE_OK`	probe หลุด / กลับมา
`USB_IN` / `USB_OUT`	เสียบ / ถอดไฟ
`POWER_ON`	เครื่องรีเซ็ตแล้วทำ trip ต่อ
`POWER_OFF`	แบตหมด เครื่องตัดตัวเอง
`TIME_SET`	มีการตั้งเวลา (จาก NTP, GNSS หรือแอปตอนเริ่ม trip)
`DATA_LOST`	trip เก่าถูกลบเพราะพื้นที่เต็ม
`SHOCK`	สำรองไว้: กระแทกแรงเกินเกณฑ์ (ยังไม่ได้ตั้งเกณฑ์)

รายละเอียดของ event (เช่นสาเหตุการรีเซ็ต, mV ตอนแบตหมด) ไม่ได้ส่งทาง MQTT อยู่ในคอลัมน์ `detail` ของไฟล์ CSV

การขยับธรรมดาไม่เป็นแถว นับรวมไว้ใน `motion`

4. ACK — สิ่งที่ server ต้องตอบ

1. เก็บแถวของชุดนั้นลงฐานข้อมูลให้เรียบร้อย **ใช้ (trip_id, seq) เป็น key** record ที่มาซ้ำต้องเก็บครั้งเดียว
2. จากนั้น publish ไปที่ `mcold/v1/<sn>/ack`

```
{"trip_id":"95518e06-1b3b-494f-b4c5-284ae94f5424","upto":15}
```

แปลว่า **record ของ trip นั้นตั้งแต่ seq 0 ถึง 15 เก็บครบแล้ว** ต้องต่อเนื่อง ห้ามตอบ `upto` ถ้ายังมี seq ที่น้อยกว่านั้นขาดอยู่

- **PUBACK ของ MQTT ไม่นับเป็น ACK** เพราะบอกแค่ว่า broker ได้รับ ไม่ใช่ว่า server เก็บแล้ว
- กล่องตรวจ ACK ทุกครั้ง: ถ้าเป็น trip ที่ไม่มี หรือ `upto` เกินข้อมูลที่มี จะไม่รับ
- ACK ซ้ำหรือมาช้าไม่เสียหาย เพราะค่านี้เดินหน้าอย่างเดียว

5. จังหวะการส่งเมื่อกล่องใช้แบต

กล่องหลับระหว่างรอบเพื่อประหยัดแบต และ Wi-Fi ปิดอยู่ จะเชื่อมต่อเป็น **รอบสั้นๆ** ทุก 5 นาที ระหว่างมี trip (ส่ง `status`) หรือเมื่อมี trip ที่จบแล้วรอส่ง:

1. เปิด Wi-Fi → เชื่อม broker → ส่ง `status` → ส่งก้อนแถวของ trip ที่จบแล้ว
2. **รอ ACK ชุดละไม่เกิน 10 วินาที** ได้ ACK แล้วส่งชุดถัดไป

3. จบรอบ: ส่ง `"0"` ที่ `online` แล้วตัดการเชื่อมต่อ

สถานการณ์	กล่องทำอะไร
server ACK ทันเวลา	รอบถัดไปตามปกติใน 5 นาที
ไม่ได้ ACK ในรอบนั้น	เว้นระยะรอบถัดไปเป็น 10, 20, 40 … นาที สูงสุด 4 ชั่วโมง
ACK มาหลังจบรอบแล้ว	ACK นั้นหายไป (กล่องใช้ clean session) รอบหน้ากล่องส่งชุดเดิมซ้ำ server แค่ ACK อีกครั้ง

สรุปสำหรับ server:

- ACK ภายใน 1–2 วินาที ทำให้แต่ละรอบใช้เวลาราว 5 วินาที
- ดูว่ากล่องยังติดต่อได้จากเวลาของ `status` ล่าสุด ไม่ใช่จาก `online` เพราะกล่องที่ใช้แบตจะเป็น `"0"` เกือบตลอดเวลาและถือว่าปกติ
- ตอนเสียบสาย USB กล่องเชื่อมต่อค้างไว้ตลอด ใช้ทดสอบฝั่ง server ได้สะดวก

6. ความปลอดภัย — ขอให้เปิดก่อนใช้งานจริง

- **TLS ที่พอร์ต 8883** พร้อม certificate ของ server ที่กล่องตรวจสอบได้
- **login แยกต่อเครื่อง** (หรือ client certificate)
- **ACL:** แต่ละกล่อง publish ได้เฉพาะ `mcold/<sn>/rec`, `status`, `online`, `ota/state` ของตัวเอง และ subscribe ได้เฉพาะ `mcold/v1/<sn>/ack`, `firmware` ของตัวเอง ส่วน `ack` และ `firmware` ให้ publish ได้เฉพาะ server

เหตุผล: ACK คือคำสั่งให้กล่อง "ลบข้อมูลได้" ถ้าใครปลอม ACK ได้ ข้อมูลที่ยังไม่ถึง server จะหายถาวร

7. คำถามที่ต้องการคำตอบ

1. ยอมรับรูปแบบ topic และ ACK ตามนี้ไหม หรืออยากปรับตรงไหน
2. เปิด TLS 8883 และ login แยกเครื่องได้เมื่อไหร่
3. ~~OTA แบบไหน~~ ตกลงแล้ว: แบบเดียวกับ eTEMP (ข้อ 8)
4. ใครเป็นผู้ติดต่อฝั่ง server สำหรับทดสอบร่วมกัน
5. broker จำกัดขนาดข้อความไว้เท่าไร (ก้อนละ 20 แถว ≈ 6 KB ต้องไม่ต่ำกว่า 8 KB)
6. แอปจะส่งไฟล์ CSV ของ trip ขึ้น server ทางไหน (HTTPS endpoint หรือ MQTT) และ server ตอบอะไรเมื่อเก็บแล้ว — แอปต้องได้คำยืนยันก่อนบอกกล่องว่า "ส่งแล้ว" (`MARK_DELIVERED`)

8. อัปเดต firmware (OTA)

ทำแบบเดียวกับ eTEMP: server ส่ง**แค่ชื่อไฟล์** กล่องดาวน์โหลดเองจาก file server (`https://drive.siamatic.co.th/media/firmwares/<ชื่อไฟล์>`) ทดสอบผ่านแล้ว 2026-10-04

1. วางไฟล์ `mCOLD_<เวอร์ชัน>.bin` ที่ทีม firmware ส่งให้ ไว้ในโฟลเดอร์ firmwares
2. publish ชื่อไฟล์ไปที่ `mcold/v1/<sn>/firmware` แบบ **retain** เช่น `mCOLD_0.7.1.bin` (กล่องที่ใช้แบตหลับอยู่เกือบตลอด จะเห็นคำสั่งตอนตื่นมาส่งข้อมูลรอบถัดไป)
3. ดูผลที่ `mcold/<sn>/ota/state` (retain):

state	ความหมาย
`downloading + pct`	กำลังดาวน์โหลด
`rebooting`	ติดตั้งเสร็จ กำลังรีสตาร์ท
`ok`	เวอร์ชันใหม่ทำงานและต่อ broker ได้แล้ว
`failed + reason`	ไม่ติดตั้ง (ไฟล์ไม่มี, ลายเซ็นไม่ผ่าน, ไม่ใช่ firmware mCOLD)
`deferred + reason`	รอก่อน: มี trip อยู่ (`trip`) หรือแบตต่ำกว่า 30% (`battery`) แล้วทำเองเมื่อพร้อม
`rolled_back`	เวอร์ชันใหม่ต่อ broker ไม่ได้ใน 3 นาที กล่องถอยกลับเวอร์ชันเดิมเอง
`skipped`	เวอร์ชันไม่ใหม่กว่าที่มีอยู่

1. ได้ `ok` แล้ว ให้ลบ retained message (publish ค่าว่างแบบ retain ไปที่ topic เดิม)

server ไม่ต้องตรวจอะไร: กล่องตรวจเองทั้งหมด (โครงการ, เวอร์ชัน, SHA-256 และ**ลายเซ็น**) ไฟล์ที่ไม่ได้เซ็นด้วย key ของทีม firmware จะถูกปฏิเสธ แม้จะวางอยู่ในโฟลเดอร์เดียวกัน เวอร์ชันที่กล่องใช้อยู่ดูได้จาก `status` → `fw.ver`

เอกสารอ้างอิง

- `firmware/PROTOCOL.md` ข้อ 6 — สเปกเต็มของ MQTT
- `firmware/src/record.h` — โครงสร้าง record ทุกชนิด

9. ตั้งค่ากล่องจาก server

publish เอกสารตั้งค่าที่ `mcold/v1/<sn>/config` แบบ **retain** (กล่องที่หลับอยู่จะได้รับตอนตื่น)

```
{"rev":3,
"config":{"upload_period_s":600,"temp_adj_c100":-50},
"wifi":{"add":[{"ssid":"Warehouse-2","pass":"...","dhcp":true},
{"ssid":"Office","pass":"...","dhcp":false,"ip":"192.168.1.50",
"gateway":"192.168.1.1","subnet":"255.255.255.0","dns":"8.8.8.8"}],
"del":["OldNet"]},
"ota_base":"https://drive.siamatic.co.th/media/firmwares"}
```

- ส่งแค่ส่วนที่จะเปลี่ยน · `rev` **ต้องเพิ่มทุกครั้ง** กล่องใช้แต่ละ rev ครั้งเดียว

- ผลกลับมาที่ `mcold/<sn>/config/state` (`ok` / `partial` / `refused`) พร้อมเหตุผลของค่าที่ไม่ได้ตั้ง
- **Wi-Fi 5 ช่อง:** `add` ใส่ `"slot":N` (1–5) เพื่อแก้ช่องนั้นโดยตรง และ `del` ใส่เลขช่องหรือชื่อ กล่องรายงานทั้ง 5 ช่องให้แอป (ดู Device Protocol)
- **Wi-Fi แบบ eTEMP:** `dhcp` true/false, `ip` , `gateway` , `subnet` , `dns` ตั้งแยกต่อเครือข่าย
- **ปรับอุณหภูมิ `temp_adj_c100`** (แบบ tempAdj ของ eTEMP) หน่วย 0.01 °C บวกเพิ่มจากค่าที่วัด
- ค่าบางตัว (รอบบันทึก, temp adjust, `alarm_dwell_s` , `alarm_hyst_c10`) เปลี่ยนไม่ได้ระหว่าง trip — ส่ง rev ใหม่หลังจบ trip
- **การเตือนอุณหภูมิเป็นค่าของเครื่อง** (ตัดสินใจ 2026-10-09): `alarm_dwell_s` = เวลาที่อุณหภูมิต้องอยู่นอกช่วงต่อเนื่องก่อนกล่องเตือน (0–3600 วินาที, 300) และ `alarm_hyst_c10` = ระยะที่ต้องกลับเข้าด้านในช่วงให้พ้นเส้นก่อนเคลียร์ (0–100 หน่วย 0.1 °C, 5) ใช้กับทุก trip ของเครื่องนั้น ค่าที่ใช้จริงของแต่ละ trip อยู่ในหัว trip (`hyst` , `dwell_s` ใน `GET_TRIP_SUMMARY`) ส่วนช่วงอุณหภูมิ `low` / `high` ยังเป็นของแต่ละ trip ที่แอปส่งตอนเริ่ม
- **MQTT broker ตั้งจาก server ไม่ได้** ตั้งที่ตัวเครื่องผ่าน console เท่านั้น
- รายการค่าทั้งหมด: [`../app/device-settings.md`](../app/device-settings.md) · แอปใช้เอกสารเดียวกันผ่าน BLE (`APPLY_CONFIG`)

⚠️ เอกสารนี้มีรหัส Wi-Fi: บนพอร์ต 1883 ที่ไม่เข้ารหัส ใครดักฟังก็เห็น — เป็นอีกเหตุผลที่ต้องเปิด TLS (ข้อ 6)

10. ค่าตั้งปัจจุบันของกล่อง → server

กล่อง publish ค่าตั้ง**ทั้งหมด**ที่ใช้อยู่ตอนนี้ที่ `mcold/<sn>/settings` แบบ **retain** (ตัดสินใจ 12 ต.ค.) server จึงรู้เสมอว่ากล่องตั้งอะไรไว้ ไม่ว่าจะแก้จากแอป (BLE), จาก server (ข้อ 9) หรือจากสาย USB

- **ส่งเมื่อไร:** ทุกครั้งที่กล่องต่อ broker ได้ และทุกครั้งที่ค่าใดค่าหนึ่งเปลี่ยน (ค่า config, Wi-Fi เพิ่ม/แก้/ลบ, `ota_base`) ตอนใช้แบตจะส่งในรอบ session ถัดไป (ตาม `upload_period_s`)
- **เนื้อหา:** ชุดเดียวกับ event `SETTINGS` ที่แอปได้ทาง BLE · รายการ Wi-Fi 5 ช่อง **ไม่มีรหัสผ่าน** · `utc` คือเวลาที่กล่องสร้างข้อความ (`null` ถ้ายังไม่รู้เวลา)
- ข้อความ retain อันล่าสุดคือค่าปัจจุบัน ไม่ต้องรวมหลายข้อความ
- ต่างจาก `config/state` (ข้อ 9): `config/state` คือผลของเอกสารที่ server ส่งมา ส่วน `settings` คือค่าที่กล่องใช้อยู่จริงทั้งหมด

```
{"config":{"sample_period_s":300,"temp_adj_c100":-50,"alarm_dwell_s":300,"alarm_hyst_c10":5,
"buzzer_enabled":1,"buzzer_volume":60,"upload_period_s":300, "...":"ทุกค่า"},
"editable":["sample_period_s","temp_adj_c100","alarm_dwell_s","..."],
"trip_locked":["sample_period_s","temp_adj_c100","alarm_dwell_s","alarm_hyst_c10","..."],
"server_rev":3,
"network":{"wifi":[{"slot":1,"ssid":"RDE_2.4GHz","dhcp":true},{"slot":2,"ssid":null},
{"slot":3,"ssid":"Warehouse","dhcp":false,"ip":"192.168.1.50","gateway":"192.168.1.1",
"subnet":"255.255.255.0","dns":"8.8.8.8"},{"slot":4,"ssid":null},{"slot":5,"ssid":null}],
"ota_base":"https://..."},
"utc":1791792252}
```

ช่อง	ความหมาย
`config`	ทุกค่าตั้งและค่าปัจจุบัน (รายการและความหมายใน [`../app/device-settings.md`](../app/device-settings.md))
`editable`	ค่าที่แอปและ server แก้ได้
`trip_locked`	ค่าที่แก้ไม่ได้ระหว่าง trip
`server_rev`	`rev` ของเอกสารตั้งค่าจาก server ฉบับล่าสุดที่กล่องใช้แล้ว
`network.wifi`	Wi-Fi 5 ช่อง: ชื่อ, DHCP หรือ IP คงที่ · ช่องว่าง `ssid` เป็น `null`
`network.ota_base`	ที่อยู่ดาวน์โหลด firmware
`utc`	เวลาที่ส่ง (วินาที Unix)